NIYOJAN V3

Agentic AI Intelligence Hub

Strategic AI Planning Assistant

---

1 Introduction

Traditional forecasting systems generate predictions but do not actively support decision-making. While demand forecasts help estimate future requirements, they do not answer critical operational questions such as:

- What happens if demand suddenly increases?
- Which products are at immediate risk?
- How should procurement be adjusted?
- What should management prioritize next month?

To address this gap, NIYOJAN V3 introduces an **Agentic AI Intelligence Hub** — a structured, deterministic, and explainable decision intelligence layer built on top of the forecasting engine.

The Intelligence Hub transforms NIYOJAN from a:

> Forecasting System

into a:

> Strategic AI-Powered Decision Planning Platform

---

2 System Objective

The Intelligence Hub is designed to:

✔ Consume forecast outputs generated by the Forecast Module
✔ Ingest analytical reports (optional)

✔ Understand natural language user queries
✔ Perform scenario simulations (e.g., +15% demand)
✔ Recalculate inventory risk
✔ Generate structured procurement strategies
✔ Provide executive-ready planning outputs

3️⃣ Overall System Architecture

🔷 Platform-Level Architecture

[Forecast Module]
↓
Stores forecast results in database
↓
[Agentic AI Intelligence Hub]
↓
LangGraph Orchestrator
↓
Deterministic Tools + RAG
↓
Strategic Planning Output

Key design principle:

- The Intelligence Hub does NOT retrain models.
- It operates strictly on forecast results.
- No training/test CSV uploads are required.

4️⃣ Internal Execution Flow

Inside the Intelligence Hub, the system follows a structured pipeline:

User Query
↓
Intent Classification
↓
Conditional Routing

↓
[Retrieval | Analysis | Simulation | Strategy]
↓
Structured Context Builder
↓
Strategic LLM Reasoning
↓
Executive Formatter

The system uses a **single orchestrated LangGraph agent** with multiple deterministic tool paths.

5️⃣ Data Layer Design

🔷 Forecast Data Schema

Stored from the Forecast Module:

| product_id | forecast_demand | std_dev | lead_time | current_stock |

Additional computed metrics:

- Volatility = std_dev / mean_demand
- Base Reorder Point
- Base Inventory Gap
- Base Risk Score

These values are precomputed to ensure performance efficiency and determinism.

🔷 Report Ingestion (RAG Layer)

Users may upload an analytical report (PDF).

Processing pipeline:

1. Extract text
2. Clean content
3. Chunk into 600–800 token segments

4. Generate embeddings
5. Store in vector database (Chroma or FAISS)

This enables context-aware explanations grounded in uploaded documentation.

6️⃣ Agent Architecture (LangGraph-Based)

The Intelligence Hub uses a **single structured agent** with conditional routing.

🔷 Agent State

```
class AgentState(TypedDict):
    query: str
    intent: str
    simulation_percent: float | None
    forecast_snapshot: dict
    rag_context: list
    analysis_result: dict
    simulation_result: dict
    strategic_context: dict
    final_response: str
```

This structured state ensures clarity and controlled execution.

7️⃣ Core Agent Nodes

7.1 Intent Classification Node

Purpose:

- Detect query type
- Extract parameters (e.g., 15% increase)

Possible intents:

- retrieval
- analysis
- simulation
- strategy

Example:

Input:

> “What if demand increases by 15%?”

Output:

- intent = simulation
- simulation_percent = 15

7.2 Retrieval Node (RAG)

Used for:

- Report explanation
- Insight clarification
- Context-based questions

Steps:

- Embed user query
- Perform similarity search
- Retrieve top-k chunks
- Store in state

No calculations performed here.

7.3 Analysis Node (Descriptive Analytics)

Handles non-simulation analytical queries.

Examples:

- Highest volatility SKU
- Largest inventory gap

- Average lead time

Pure Python computations:

volatility = std_dev / mean_demand
gap = reorder_point - current_stock

Outputs structured analysis results.

No LLM used for computation.

7.4 Simulation Node (Deterministic Counterfactual Engine)

Handles scenario-based queries.

Example:

> “What if demand increases by 15%?”

Step 1 – Adjust Demand

new_demand = forecast_demand × (1 + percent/100)

Step 2 – Recalculate Safety Stock

SafetyStock = Z × std_dev × √lead_time

Step 3 – Recalculate Reorder Point

ROP = (new_demand × lead_time) + SafetyStock

Step 4 – Compute Inventory Gap

Gap = ROP - current_stock

Step 5 – Assign Risk Level

Example thresholds:

- Gap > 200 → High Risk
- Gap 100–200 → Medium Risk
- Else → Low Risk

This node is fully deterministic and avoids hallucination.

7.5 Strategy Node (Planning Context Builder)

This node prepares structured input for strategic reasoning.

It aggregates:

- High-risk SKUs
- Gap magnitudes
- Volatility metrics
- Lead-time sensitivity
- Simulation results (if applicable)
- RAG context (if needed)

This structured context is passed to the LLM.

7.6 Strategic Reasoning Node (LLM)

Role of LLM:

- Synthesize structured inputs
- Generate executive strategy
- Prioritize SKUs
- Provide 4-week plan

The LLM is NOT used for calculations.

7.7 Executive Formatter

Ensures consistent structured output:

📊 Executive Summary

⚠ High-Risk Products

📈 Key Risk Drivers

🛒 Recommended Actions

📅 4-Week Action Plan

📌 Monitoring Strategy

8️⃣ End-to-End Scenario Example

User Query:

> “What if demand increases by 15% next month?”

Execution:

1. Intent classifier detects simulation
2. Extract 15%
3. Fetch forecast snapshot
4. Run deterministic simulation
5. Recalculate risk metrics
6. Identify top high-risk SKUs
7. Strategy Node builds structured context
8. LLM generates procurement plan
9. Executive formatter outputs structured report

Flow:

Forecast → Simulation → Risk Recalculation → Strategic Planning

---

9️⃣ Backend Architecture

backend/
 ├── agents/

```
│   ├── graph.py
│   ├── state.py
│   ├── intent.py
│   ├── nodes/
│       ├── retrieval.py
│       ├── analysis.py
│       ├── simulation.py
│       ├── strategy.py
│       ├── formatter.py
│
├── tools/
│   ├── inventory_math.py
│   ├── risk_metrics.py
│
├── rag/
│   ├── ingestion.py
│   ├── vector_store.py
│
├── services/
│   ├── forecast_service.py
│
├── routes/
│   ├── intelligence.py
```

This separation ensures:

- Maintainability
- Scalability
- Clear research documentation
- Modular testing

🔟 Research Contributions

The Agentic AI Intelligence Hub contributes:

1. Single-agent conditional orchestration using LangGraph
2. Deterministic scenario simulation for inventory planning
3. Hybrid RAG + structured planning architecture
4. Risk-aware procurement decision modeling
5. Explainable AI for operational planning

1️⃣1️⃣ Why This System Is Strong

✔ Deterministic calculations prevent hallucination
✔ LLM is restricted to reasoning, not math
✔ Clear separation of responsibilities
✔ Modular agent design
✔ Scalable backend structure
✔ Research-grade architecture

1️⃣2️⃣ Final System Identity

NIYOJAN V3 becomes:

> Agentic AI-Powered Strategic Planning Platform
> Combining Forecasting, Deterministic Simulation, Risk Modeling, and Intelligent Procurement Planning.

It is:

- Technically sound
- Architecturally clean
- Academically publishable
- Industry-relevant
- Internship-ready
- Startup-grade

If you want, I can now:

- Convert this into a formal research paper section
- Convert this into a polished PDF report structure
- Provide implementation-ready LangGraph code
- Design your final strategic LLM prompt

We are now at production-level clarity.